MiniC Language Specification

Project: AI-Based Compiler Optimization Recommendation System
Scope: Middle-ground feature set — locked for all 4 team members

---

1. Design Philosophy

Every rule below exists to serve one invariant: **no aliasing is possible in MiniC.** No pointers means no two variables can ever refer to the same memory, which means your optimization passes (constant folding, DCE, CSE, LICM, strength reduction) can be proven correct without alias analysis — a problem that's genuinely research-level on its own.

Break this invariant anywhere, even accidentally through codegen, and your "measured speedup" numbers stop being trustworthy — the optimized binary could produce different output than the unoptimized one. Every feature added below (structs, 2D arrays, fixed strings) was chosen because it preserves this invariant. Read Section 4

carefully — it's the one place this is easy to break by accident.

2. Feature Set (Locked)

Included	Excluded
`int`, `float`, `char`	Pointers (any form)
1D and 2D fixed-size arrays	Dynamic memory (`malloc` / `free`)
Structs (value-only, non-recursive)	Recursive/self-referential structs
Fixed-size strings (char arrays)	Variable-length strings
Functions, recursion	Function pointers, variadics
`if/else`, `while`, `for`	`goto`, `switch`
Block scoping	Preprocessor macros
Arithmetic/relational/logical ops	Unions, bitfields
	Runtime array bounds checking

Included	Excluded
	(compile-time size checks only)

3. Type System

Primitives: `int`, `float`, `char`

Arrays: `T name[N]` (1D) or `T name[M][N]` (2D). N and M must be integer literals, fixed at compile time — no variable-length arrays.

Structs: declared with `struct Name { ... }`. Fields may be primitives, fixed arrays, or other struct types — but a struct can never contain itself, directly or through a chain of other structs (that would require a pointer to have a well-defined size, exactly what we're avoiding). Structs are typed **nominally** — two structs with identical fields but different names are different types.

Strings: sugar for `char name[N]`. A string literal `"hello"` is valid anywhere a `char[N]` is expected, with N ≥ length + 1 (for the null terminator), checked at compile time.

Type compatibility: arrays are compatible only if element type and every dimension match exactly. Structs are compatible only if they share the same declared name.

4. Semantics — The Value Rule (read before writing codegen)

Assignment, function parameter passing, and return values are **always copy semantics** for arrays, strings, and structs. No exceptions.

The codegen trap: real C arrays decay to pointers the moment you pass them to a function — `int a[10]; int b[10] = a;` doesn't even compile in real C, and passing an array to a function passes a pointer, not a copy. If your codegen emits MiniC arrays as literal C arrays, you will silently reintroduce aliasing the instant two MiniC variables share array data through a function call — even though the MiniC source never used a pointer.

The fix: wrap every array and string type in a one-field C struct at codegen time.

```
// MiniC: int scores[10];
// Emit as:
typedef struct { int data[10]; }
arr_int_10;
arr_int_10 scores;
```

Because C structs (unlike raw arrays) are copied by value on assignment and on function call, this wrapper forces true copy semantics in the generated C. **Whoever owns codegen must**

implement this for every array/string declaration, with no exceptions — otherwise every downstream optimization-correctness claim in the report is unsound.

Practical trade-off worth a line in your limitations section: passing large arrays/structs to functions means a full copy each call. For a semester project's benchmark sizes this is fine, and it's the same trade-off real languages accept for soundness without alias analysis — it's part of why Rust's borrow checker exists, to get pointer-like efficiency without losing this guarantee.

5. Grammar (BNF)

```
program         → (struct_decl | func_decl
| var_decl)*

struct_decl     → 'struct' IDENT '{'
field_decl+ '}' ';'
field_decl      → type_spec IDENT
array_suffix? ';'

var_decl        → type_spec IDENT
array_suffix? ('=' expr)? ';'
array_suffix    → '[' INT_LIT ']' ('['
INT_LIT ']')?

type_spec       → 'int' | 'float' | 'char'
| IDENT        // IDENT = struct name
```

```
func_decl       → type_spec IDENT '('
param_list? ')' block
param_list      → param (',' param)*
param           → type_spec IDENT
array_suffix?

block           → '{' stmt* '}'
stmt            → var_decl | expr_stmt |
if_stmt | while_stmt
                 | for_stmt | return_stmt |
block

if_stmt         → 'if' '(' expr ')' stmt
('else' stmt)?
while_stmt      → 'while' '(' expr ')'
stmt
for_stmt        → 'for' '(' for_init ';'
expr ';' expr ')' stmt
for_init        → type_spec IDENT '=' expr
| expr | ε
return_stmt     → 'return' expr? ';'
expr_stmt       → expr ';'

expr            → assignment
assignment      → postfix '=' assignment |
logic_or
logic_or        → logic_and ('||'
logic_and)*
logic_and       → equality ('&&'
equality)*
equality        → relational (('==' |
'!=') relational)*
relational      → additive (('<' | '>' |
```

```
'<=' | '>=') additive)*
additive       → multiplicative (('+' |
'-') multiplicative)*
multiplicative  → unary (('*' | '/' | '%')
unary)*
unary          → ('-' | '!') unary |
postfix
postfix        → primary ('[' expr ']' |
'.' IDENT | '(' arg_list? ')')*
primary        → INT_LIT | FLOAT_LIT |
CHAR_LIT | STRING_LIT
                | IDENT | '(' expr ')'
arg_list       → expr (',' expr)*
```

Note: the grammar allows any `postfix` on the left of `=`. Semantic analysis — not the parser — is responsible for rejecting invalid lvalues like `f(x) = 5` or `3 = x`.

6. Canonical Example Program

Give every phase owner this program on day one. It exercises structs, 2D arrays, a fixed string, recursion, and control flow in one place, so P1/P2/P3/P4 are all testing against the same ground truth.

```
struct Point {
    int x;
    int y;
};
```

```
int matrixSum(int m[3][3]) {
    int total = 0;
    int i = 0;
    while (i < 3) {
        int j = 0;
        while (j < 3) {
            total = total + m[i][j];
            j = j + 1;
        }
        i = i + 1;
    }
    return total;
}

int distanceSquared(struct Point a, struct
Point b) {
    int dx = a.x - b.x;
    int dy = a.y - b.y;
    return dx * dx + dy * dy;
}

int fib(int n) {
    if (n < 2) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

int main() {
    char label[6] = "grid1";
    struct Point p1;
    p1.x = 0;
    p1.y = 0;
```

```
    struct Point p2;
    p2.x = 3;
    p2.y = 4;

    int grid[3][3];
    grid[0][0] = 1; grid[0][1] = 0; grid[0]
[2] = 0;
    grid[1][0] = 0; grid[1][1] = 1; grid[1]
[2] = 0;
    grid[2][0] = 0; grid[2][1] = 0; grid[2]
[2] = 1;

    int d = distanceSquared(p1, p2);
    int s = matrixSum(grid);
    int f = fib(10);

    return d + s + f;
}
```

Expected output: `83` (`distanceSquared` = 25, `matrixSum` = 3, `fib(10)` = 55). Use this as your first correctness checkpoint once the front-end, codegen, and gcc compilation are wired together — before worrying about optimizations or timing at all, confirm the unoptimized pipeline produces exactly `83`.

7. Impact on Feature Extraction (P1) and Benchmarks (P3)

New static features worth extracting because of this expanded scope:

- Struct field count and nesting depth (a proxy for how much CSE/copy propagation could help)
- Array dimensionality (1D vs 2D) and total element count (loops over 2D arrays are prime LICM/strength-reduction territory)
- String/char-array operation count

Benchmark corpus should include at least one program stressing each: a 2D numeric benchmark (matrix multiply, convolution), a struct-heavy benchmark (geometry/record processing), and a string-handling benchmark (reverse, palindrome check). Without all three, the dataset won't have enough feature variance for P4's model to learn anything meaningful from the struct/array-specific features.

8. Scope Lock

This document is the frozen spec. Any feature request beyond it (pointers, dynamic memory, unions, etc.) needs an explicit team conversation before anyone implements it. The timeline estimates given earlier assumed exactly this scope — quietly expanding it mid-implementation is the most

common way a semester compiler project runs out of time.